Homework 1

The goal of this exercise is to implement the backpropagation algorithm for neural networks containing only fully connected layers. Your code should be implemented over PyTorch, but you should implement the derivatives and the algorithms only using tensors. **Do not use the built-in derivation** of PyTorch.

The algorithm should get:

- The number and dimension of each layer.
- The type of non-linearity for each layer. You should support the following types of non-linearity:
    - Identity function (so the outcome is simply the linear combination plus bias)
    - ReLU
    - Sigmoid
- The error function. You Should support:
    - RSS, namely square error. For those of you who are not sure it is the error used in linear regression
    - Absolute error. Similar to the above but using absolute value instead of square loss.
    - Log-loss, also called cross entropy. This is the error used logistic regression.
- The point for evaluation. Note this contains both X and y

The output of the algorithm should be the following:

- The point of evaluation, both X and y.
- The output of the network.
- The error.
- The derivative of the error on each of the parameters.

In order for the algorithm to run, you should set an initial value for the parameters. Please initiate the weights randomly from a standard normal distribution and the biases to be 0. Once initialized keep all the weights in order to be able to debug the code. Otherwise you wouldn't be able to get the same results on consecutive runs.

Note, you may restrict your network to support only regression and 2-class classification. No need to support k-class classification and hence you don't need to implement softmax.